연결 설정 및 업그레이드 과정 문서화

개요

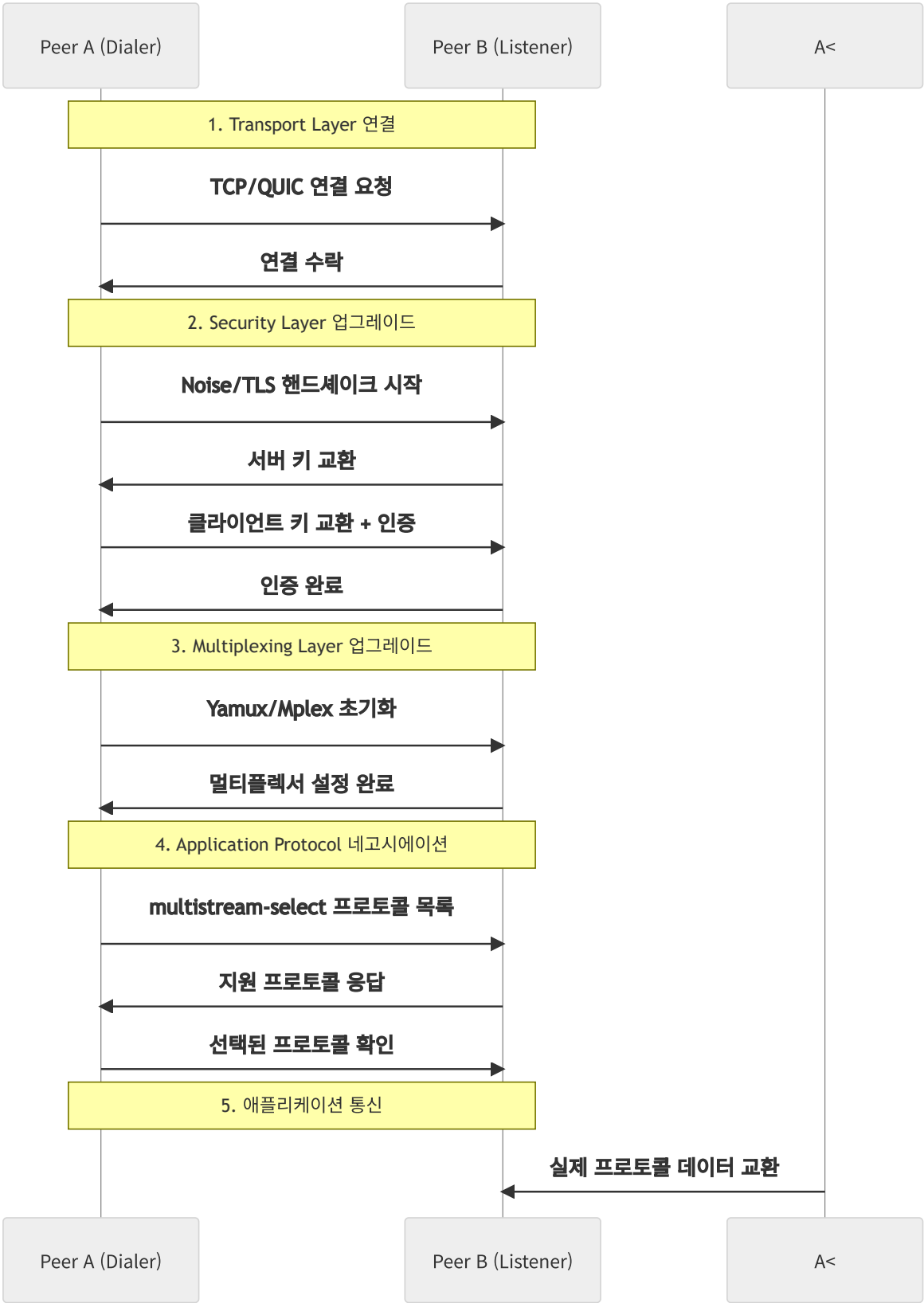
이 문서는 Rust libp2p에서 피어 간 연결이 어떻게 설정되고 프로토콜 업그레이드가 어떻게 수행되는지에 대한 상세한 분석을 제공합니다. libp2p의 연결 설정은 여러 계층을 거쳐 단계적으로 이루어지며, 각 단계에서 특정한 프로토콜 업그레이드가 수행됩니다.

연결 설정 전체 흐름

1. 연결 설정 단계별 개요

```
1. Transport Layer 연결 설정
↓
2. Security Layer 업그레이드 (인증 및 암호화)
↓
3. Multiplexing Layer 업그레이드 (스트림 멀티플렉싱)
↓
4. Application Protocol 네고시에이션
↓
5. 실제 애플리케이션 프로토콜 통신
```

2. 연결 설정 시퀀스 다이어그램


Peer A (Dialer)
Peer B (Listener)
A<
1. Transport Layer 연결
TCP/QUIC 연결 요청
연결 수락
2. Security Layer 업그레이드
Noise/TLS 핸드셰이크 시작
서버 키 교환
클라이언트 키 교환 + 인증
인증 완료
3. Multiplexing Layer 업그레이드
Yamux/Mplex 초기화
멀티플렉서 설정 완료
4. Application Protocol 네고시에이션
multistream-select 프로토콜 목록
지원 프로토콜 응답
선택된 프로토콜 확인
5. 애플리케이션 통신
실제 프로토콜 데이터 교환
Peer A (Dialer)
Peer B (Listener)
A<

Transport Layer 연결 설정

1. Transport 트레이트 구조

```
pub trait Transport {
type Output; // 연결 성공 시 반환 타입
type Error: Error; // 에러 타입
type ListenerUpgrade: Future<Output = Result<Self::Output, Self::Error>>;
type Dial: Future<Output = Result<Self::Output, Self::Error>>;

// 리스너 관리
fn listen_on(&mut self, id: ListenerId, addr: Multiaddr) -> Result<(), Transpor
fn remove_listener(&mut self, id: ListenerId) -> bool;
// 연결 설정
fn dial(&mut self, addr: Multiaddr, opts: DialOpts) -> Result<Self::Dial, Trans
// 이벤트 폴링
fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<TransportEvent<Self
}
```

2. 다이얼링 과정 (Outbound Connection)

2.1 DialOpts 구성

```
pub struct DialOpts {
```

```
pub role: Endpoint, // Dialer 또는 Listener 역할
pub port_use: PortUse, // 포트 사용 정책
}

pub  enum PortUse {
New, // 새로운 포트 할당
Reuse, // 기존 리스너 포트 재사용 (홀 펀칭용)
}
```

2.2 TCP 다이얼링 구현 예시

```
fn dial(&mut  self, addr: Multiaddr, opts: DialOpts) -> Result<Self::Dial, Trans
// 1. Multiaddr를 SocketAddr로 변환
let socket_addr = multiaddr_to_socketaddr(addr.clone())
.map_err(|_| TransportError::MultiaddrNotSupported(addr))?;
// 2. 소켓 생성 및 설정
let socket = self.config.create_socket(socket_addr, opts.port_use)
.map_err(TransportError::Other)?;
// 3. 포트 재사용을 위한 바인드 주소 결정
let bind_addr = match  self.port_reuse.local_dial_addr(&socket_addr.ip()) {
Some(addr) if opts.port_use == PortUse::Reuse => Some(addr),
_ => None,
};
// 4. 비동기 연결 Future 반환
```

```
Ok(async  move {
// 필요시 로컬 주소에 바인드 (홀 펀칭용)
if  let Some(bind_addr) = bind_addr {
socket.bind(&bind_addr.into())?;
}
// 비동기 연결 시도
match socket.connect(&socket_addr.into()) {
Ok(()) => {}, // 즉시 연결 성공
Err(err) if err.raw_os_error() == Some(libc::EINPROGRESS) => {}, // 진행 중
Err(err) if err.kind() == io::ErrorKind::WouldBlock => {}, // 블로킹 방지
Err(err) => return Err(err),
}
let stream = T::new_stream(socket.into()).await?;
Ok(stream)
}.boxed())
}
```

3. 리스닝 과정 (Inbound Connection)

3.1 리스너 설정

```
fn listen_on(&mut  self, id: ListenerId, addr: Multiaddr) -> Result<(), Transpor
let socket_addr = multiaddr_to_socketaddr(addr.clone())
```

```
.map_err(|_| TransportError::MultiaddrNotSupported(addr))?;

// 1. 리스너 소켓 생성

let socket = self.config.create_socket(socket_addr, PortUse::New)?;

// 2. 주소에 바인드

socket.bind(&socket_addr.into())?;

// 3. 리스닝 시작

socket.listen(self.config.backlog as i32)?;

// 4. 포트 재사용 풀에 등록

self.port_reuse.register(socket_addr.ip(), socket_addr.port());

// 5. 리스너 스트림 생성 및 등록

let listener = T::new_listener(socket.into())?;

let listen_stream = ListenStream::new(id, socket_addr, listener, self.port_reuse

self.listeners.push(listen_stream);

Ok(())

}
```

3.2 연결 수락 과정

```
impl<T> Stream for ListenStream<T> {

type Item = TransportEvent<Ready<Result<T::Stream, io::Error>>, io::Error>;

fn poll_next(mut  self: Pin<&mut  Self>, cx: &mut Context) -> Poll<Option<Self::

// 1. 네트워크 인터페이스 변경 이벤트 처리

if  let Poll::Ready(event) = self.poll_if_addr(cx) {
```

```
return Poll::Ready(Some(event));
}
// 2. 새로운 연결 수락
match T::poll_accept(&mut  self.listener, cx) {
Poll::Ready(Ok(Incoming { local_addr, remote_addr, stream })) => {
let local_addr = ip_to_multiaddr(local_addr.ip(), local_addr.port());
let remote_addr = ip_to_multiaddr(remote_addr.ip(), remote_addr.port());
Poll::Ready(Some(TransportEvent::Incoming {
listener_id: self.listener_id,
upgrade: future::ok(stream), // 업그레이드 Future
local_addr,
send_back_addr: remote_addr,
}))
}
Poll::Ready(Err(error)) => {
// 비치명적 오류 처리
Poll::Ready(Some(TransportEvent::ListenerError {
listener_id: self.listener_id,
error,
}))
}
Poll::Pending => Poll::Pending,
}
```

```
}

}
```

프로토콜 업그레이드 시스템

1. 업그레이드 빌더 패턴

libp2p는 타입 안전한 업그레이드 체인을 구성하기 위해 빌더 패턴을 사용합니다:

```
let transport = tcp::Transport::default()

.upgrade(upgrade::Version::V1)

.authenticate(noise::Config::new(&keypair)?)

.multiplex(yamux::Config::default())

.boxed();
```

2. 업그레이드 단계별 분석

2.1 인증 업그레이드 (Authentication)

```
impl<T> Builder<T> {

pub  fn authenticate<C, D, U, E>(

self,

upgrade: U,

) -> Authenticated<AndThen<T, impl FnOnce(C, ConnectedPoint) -> Authenticate<C,

where
```

```
T: Transport<Output = C>,

C: AsyncRead + AsyncWrite + Unpin,

D: AsyncRead + AsyncWrite + Unpin,

U: InboundConnectionUpgrade<Negotiated<C>, Output = (PeerId, D), Error = E>,

U: OutboundConnectionUpgrade<Negotiated<C>, Output = (PeerId, D), Error = E> + C

E: Error + 'static,

{

// 인증 업그레이드를 Transport에 적용

let version = self.version;

Authenticated(Builder::new(

self.inner.and_then(move |conn, endpoint| Authenticate {

inner: upgrade::apply(conn, upgrade, endpoint, version),

}),

version,

))

}

}
```

인증 업그레이드의 특징:

- **입력**: 원시 연결 ©
- **출력**: (PeerId, 인증된 연결) 튜플
- **역할**: 피어 신원 확인 및 보안 채널 설정

2.2 멀티플렉싱 업그레이드 (Multiplexing)

```rust
impl<T> Authenticated<T> {
pub  fn multiplex<C, M, U, E>(
self,
upgrade: U,
) -> Multiplexed<AndThen<T, impl FnOnce((PeerId, C), ConnectedPoint) -> Multiple
where
T: Transport<Output = (PeerId, C)>,
C: AsyncRead + AsyncWrite + Unpin,
M: StreamMuxer,
U: InboundConnectionUpgrade<Negotiated<C>, Output = M, Error = E>,
U: OutboundConnectionUpgrade<Negotiated<C>, Output = M, Error = E> + Clone,
E: Error + 'static,
{
// 멀티플렉싱 업그레이드 적용
let version = self.0.version;
Multiplexed(self.0.inner.and_then(move |(peer_id, conn), endpoint| {
let upgrade = upgrade::apply(conn, upgrade, endpoint, version);
Multiplex {
peer_id: Some(peer_id),
upgrade,
}
}))
```

```
}

}
```

멀티플렉싱 업그레이드의 특징:

- **입력**: (PeerId, 인증된 연결) 튜플
- **출력**: (PeerId, StreamMuxer) 튜플
- **역할**: 단일 연결에서 다중 스트림 지원

3. 업그레이드 적용 과정

3.1 upgrade::apply 함수

```
pub(crate) fn apply<C, U>(
conn: C,
upgrade: U,
endpoint: ConnectedPoint,
version: Version,
) -> EitherUpgrade<C, U>
where
C: AsyncRead + AsyncWrite + Unpin,
U: InboundConnectionUpgrade<Negotiated<C>> + OutboundConnectionUpgrade<Negotiate
{
match endpoint {
ConnectedPoint::Dialer { .. } => {
// 아웃바운드 연결: 클라이언트 역할
```

```
future::Either::Right(apply_outbound(conn, upgrade, version))
}
ConnectedPoint::Listener { .. } => {
// 인바운드 연결: 서버 역할
future::Either::Left(apply_inbound(conn, upgrade))
}
}
}
```

3.2 인바운드 업그레이드 적용

```
pub(crate) fn apply_inbound<C, U>(
conn: C,
upgrade: U,
) -> InboundUpgradeApply<C, U>
where
C: AsyncRead + AsyncWrite + Unpin,
U: InboundConnectionUpgrade<Negotiated<C>>,
{
InboundUpgradeApply {
state: InboundUpgradeApplyState::Init {
future: multistream_select::listener_select_proto(conn, upgrade),
},
```

```
}

}
```

3.3 아웃바운드 업그레이드 적용

```
pub(crate) fn apply_outbound<C, U>(

conn: C,

upgrade: U,

version: Version,

) -> OutboundUpgradeApply<C, U>

where

C: AsyncRead + AsyncWrite + Unpin,

U: OutboundConnectionUpgrade<Negotiated<C>>,

{

OutboundUpgradeApply {

state: OutboundUpgradeApplyState::Init {

future: multistream_select::dialer_select_proto(conn, upgrade, version),

},

}

}
```

보안 계층 업그레이드 (Noise Protocol)

1. Noise 설정 구조

```rust
#[derive(Clone)]
pub struct Config {
dh_keys: AuthenticKeypair, // Diffie-Hellman 키 쌍
params: NoiseParams, // Noise 프로토콜 매개변수
webtransport_certhashes: Option<HashSet<Multihash<64>>>, // WebTransport 인증서 ㅎ
prologue: Vec<u8>, // 핸드셰이크 프롤로그
}

impl Config {
pub fn new(identity: &identity::Keypair) -> Result<Self, Error> {
let noise_keys = Keypair::new().into_authentic(identity)?;
Ok(Self {
dh_keys: noise_keys,
params: PARAMS_XX.clone(), // XX 핸드셰이크 패턴 사용
webtransport_certhashes: None,
prologue: vec![],
})
}
}
```

2. Noise 핸드셰이크 과정

2.1 XX 핸드셰이크 패턴

```
Initiator (Dialer) Responder (Listener)

| |

| ---- (empty message) ----> | (1단계: 빈 메시지)

| |

| <--- (identity) ---------- | (2단계: 응답자 신원)

| |

| ---- (identity) ---------> | (3단계: 개시자 신원)

| |

| 암호화된 통신 시작 |
```

2.2 인바운드 업그레이드 (Responder)

```rust
impl<T> InboundConnectionUpgrade<T> for Config

where

T: AsyncRead + AsyncWrite + Unpin + Send + 'static,

{

type Output = (PeerId, Output<T>);

type Error = Error;

type Future = Pin<Box<dyn Future<Output = Result<Self::Output, Self::Error>> + S


fn upgrade_inbound(self, socket: T, _: Self::Info) -> Self::Future {

async  move {

let  mut state = self.into_responder(socket)?;
```

```
// XX 핸드셰이크 단계별 실행
handshake::recv_empty(&mut state).await?; // 1. 빈 메시지 수신
handshake::send_identity(&mut state).await?; // 2. 신원 정보 전송
handshake::recv_identity(&mut state).await?; // 3. 상대방 신원 수신

let (pk, io) = state.finish()?;
Ok((pk.to_peer_id(), io))
}
.boxed()
}
}
```

2.3 아웃바운드 업그레이드 (Initiator)

```
impl<T> OutboundConnectionUpgrade<T> for Config
where
T: AsyncRead + AsyncWrite + Unpin + Send + 'static,
{
type Output = (PeerId, Output<T>);
type Error = Error;
type Future = Pin<Box<dyn Future<Output = Result<Self::Output, Self::Error>> + S
```

```rust
fn upgrade_outbound(self, socket: T, _: Self::Info) -> Self::Future {
async move {
let mut state = self.into_initiator(socket)?;

// XX 핸드셰이크 단계별 실행
handshake::send_empty(&mut state).await?; // 1. 빈 메시지 전송
handshake::recv_identity(&mut state).await?; // 2. 상대방 신원 수신
handshake::send_identity(&mut state).await?; // 3. 신원 정보 전송

let (pk, io) = state.finish()?;
Ok((pk.to_peer_id(), io))
}
.boxed()
}
}
```

3. 핸드셰이크 상태 관리

```rust
pub struct State<T> {
socket: T, // 기본 소켓
session: snow::HandshakeState, // Noise 핸드셰이크 상태
identity: identity::Keypair, // 로컬 신원
remote_identity: Option<identity::PublicKey>, // 원격 신원
```

```
webtransport_certhashes: Option<HashSet<Multihash<64>>>,
}


impl<T> State<T> {
pub fn finish(mut self) -> Result<(identity::PublicKey, Output<T>), Error> {
// 핸드셰이크 완료 및 전송 상태로 전환
let session = self.session.into_transport_mode()?;
let remote_identity = self.remote_identity.take()
.ok_or(Error::AuthenticationFailed)?;
let io = Output::new(self.socket, session);
Ok((remote_identity, io))
}
}
```

멀티플렉싱 계층 업그레이드

1. Yamux 멀티플렉서 설정

```
#[derive(Debug, Clone)]
pub struct Config {
max_buffer_size: usize, // 최대 버퍼 크기
max_num_streams: usize, // 최대 스트림 수
window_update_mode: WindowUpdateMode, // 윈도우 업데이트 모드
```

```
read_after_close: bool, // 종료 후 읽기 허용
}

impl Default for Config {
fn default() -> Self {
Config {
max_buffer_size: 256 * 1024, // 256KB
max_num_streams: 512, // 512개 스트림
window_update_mode: WindowUpdateMode::OnReceive,
read_after_close: true,
}
}
}
```

2. Yamux 업그레이드 구현

```
impl<C> InboundConnectionUpgrade<C> for Config
where
C: AsyncRead + AsyncWrite + Unpin + Send + 'static,
{
type Output = Muxer<C>;
type Error = io::Error;
type Future = future::Ready<Result<Self::Output, Self::Error>>;
```

```
fn upgrade_inbound(self, socket: C, _: Self::Info) -> Self::Future {
let config = self.into_yamux_config();
let connection = yamux::Connection::new(socket, config, yamux::Mode::Server);
future::ok(Muxer::new(connection))
}
}

impl<C> OutboundConnectionUpgrade<C> for Config
where
C: AsyncRead + AsyncWrite + Unpin + Send + 'static,
{
type Output = Muxer<C>;
type Error = io::Error;
type Future = future::Ready<Result<Self::Output, Self::Error>>;

fn upgrade_outbound(self, socket: C, _: Self::Info) -> Self::Future {
let config = self.into_yamux_config();
let connection = yamux::Connection::new(socket, config, yamux::Mode::Client);
future::ok(Muxer::new(connection))
}
}
```

3. StreamMuxer 구현

```
impl<C> StreamMuxer for Muxer<C>

where

C: AsyncRead + AsyncWrite + Unpin,

{

type Substream = Stream;

type Error = yamux::ConnectionError;


fn poll_inbound(&mut  self, cx: &mut Context<'_>) -> Poll<Result<Self::Substream

// 인바운드 스트림 수락

self.connection.poll_next_inbound(cx)

.map(|result| result.map(Stream::new))

}


fn poll_outbound(&mut  self, cx: &mut Context<'_>) -> Poll<Result<Self::Substrea

// 아웃바운드 스트림 생성

self.connection.poll_open_stream(cx)

.map(|result| result.map(Stream::new))

}


fn poll_close(&mut  self, cx: &mut Context<'_>) -> Poll<Result<(), Self::Error>>

// 연결 종료
```

```
self.connection.poll_close(cx)

}

}
```

프로토콜 네고시에이션 (multistream-select)

1. 프로토콜 정보 제공

```
pub  trait UpgradeInfo {

type Info: AsRef<str> + Clone;

type InfoIter: IntoIterator<Item = Self::Info>;


fn protocol_info(&self) -> Self::InfoIter;

}


// 예시: Noise 프로토콜

impl UpgradeInfo for noise::Config {

type Info = &'static str;

type InfoIter = std::iter::Once<Self::Info>;


fn protocol_info(&self) -> Self::InfoIter {

std::iter::once("/noise")

}
```

```
}
```

2. 프로토콜 선택 과정

2.1 Dialer (클라이언트) 측

```
pub  async  fn dialer_select_proto<R, I, C>(
inner: R,
protocols: I,
version: Version,
) -> Result<(I::Item, NegotiatedComplete<R>), NegotiationError>
where
R: AsyncRead + AsyncWrite + Unpin,
I: IntoIterator,
I::Item: AsRef<str>,
{
// 1. multistream-select 헤더 전송
let  mut inner = NegotiatedComplete::new(inner);
match version {
Version::V1 => {
write_one(&mut inner, b"/multistream/1.0.0").await?;
}
}
// 2. 지원 프로토콜 목록 전송
```

```
for protocol in protocols {
write_one(&mut inner, protocol.as_ref().as_bytes()).await?;
// 3. 서버 응답 대기
let response = read_one(&mut inner).await?;
if response == protocol.as_ref().as_bytes() {
// 프로토콜 선택 성공
return Ok((protocol, inner));
}
}
Err(NegotiationError::Failed)
}
```

2.2 Listener (서버) 측

```
pub  async  fn listener_select_proto<R, I>(
inner: R,
protocols: I,
) -> Result<(I::Item, NegotiatedComplete<R>), NegotiationError>
where
R: AsyncRead + AsyncWrite + Unpin,
I: IntoIterator,
I::Item: AsRef<str>,
{
let  mut inner = NegotiatedComplete::new(inner);
```

```
let protocols: Vec<_> = protocols.into_iter().collect();

// 1. multistream-select 헤더 수신 대기

let header = read_one(&mut inner).await?;

if header != b"/multistream/1.0.0" {

return Err(NegotiationError::Failed);

}

// 2. 클라이언트 프로토콜 요청 처리

loop {

let request = read_one(&mut inner).await?;

// 지원하는 프로토콜인지 확인

for protocol in &protocols {

if request == protocol.as_ref().as_bytes() {

// 프로토콜 지원 확인 응답

write_one(&mut inner, protocol.as_ref().as_bytes()).await?;

return Ok((protocol.clone(), inner));

}

}

// 지원하지 않는 프로토콜

write_one(&mut inner, b"na").await?;

}

}
```

실제 연결 설정 예시

1. SwarmBuilder를 통한 전체 스택 구성

```
use libp2p::{noise, tcp, yamux, SwarmBuilder};


#[tokio::main]
async fn main() -> Result<(), Box<dyn Error>> {
let mut swarm = SwarmBuilder::with_new_identity()
.with_tokio()
.with_tcp(
tcp::Config::default(), // TCP Transport
noise::Config::new, // Noise 보안 계층
yamux::Config::default, // Yamux 멀티플렉싱
)?
.with_behaviour(|_| ping::Behaviour::default())? // Ping 프로토콜
.build();


// 리스너 시작
swarm.listen_on("/ip4/0.0.0.0/tcp/0".parse()?)?;


// 원격 피어에 연결
if let Some(addr) = std::env::args().nth(1) {
```

```
let remote: Multiaddr = addr.parse()?;

swarm.dial(remote)?;

}



// 이벤트 루프

loop {

match swarm.select_next_some().await {

SwarmEvent::NewListenAddr { address, .. } => {

println!("Listening on {address:?}");

}

SwarmEvent::Behaviour(event) => {

println!("{event:?}");

}

_ => {}

}

}

}
```

2. 수동 Transport 구성 예시

```
use libp2p_core::{transport::MemoryTransport, upgrade, Transport};

use libp2p_identity as identity;

use libp2p_noise as noise;
```

```
use libp2p_yamux as yamux;


fn build_transport() -> Result<impl Transport<Output = (PeerId, impl StreamMuxer

let id_keys = identity::Keypair::generate_ed25519();

let transport = tcp::Transport::default()

.upgrade(upgrade::Version::V1)

.authenticate(noise::Config::new(&id_keys)?)

.multiplex(yamux::Config::default())

.boxed();

Ok(transport)

}
```

연결 설정 최적화 및 오류 처리

1. 연결 타임아웃 설정

```
let transport = tcp::Transport::default()

.upgrade(upgrade::Version::V1)

.authenticate(noise::Config::new(&id_keys)?)

.multiplex(yamux::Config::default())

.timeout(Duration::from_secs(20)) // 전체 업그레이드 타임아웃

.boxed();
```

2. 포트 재사용을 통한 NAT 트래버설

```
// 다이얼링 시 포트 재사용 옵션 설정
let dial_opts = DialOpts {
role: Endpoint::Dialer,
port_use: PortUse::Reuse, // 기존 리스너 포트 재사용
};

transport.dial(remote_addr, dial_opts)?;
```

3. 오류 처리 전략

```
match transport.dial(addr, opts) {
Ok(dial_future) => {
match dial_future.await {
Ok((peer_id, muxer)) => {
println!("Connected to {peer_id}");
// 연결 성공 처리
}
Err(TransportUpgradeError::Transport(e)) => {
eprintln!("Transport error: {e}");
// Transport 계층 오류 처리
}
```

```
Err(TransportUpgradeError::Upgrade(e)) => {
eprintln!("Upgrade error: {e}");
// 프로토콜 업그레이드 오류 처리
}
}
}
Err(TransportError::MultiaddrNotSupported(addr)) => {
eprintln!("Unsupported address: {addr}");
// 지원하지 않는 주소 형식
}
Err(TransportError::Other(e)) => {
eprintln!("Other transport error: {e}");
// 기타 Transport 오류
}
}
```

성능 고려사항

1. 연결 풀링

- **연결 재사용**: 동일한 피어에 대한 다중 연결 방지
- **연결 유지**: idle 연결 타임아웃 설정으로 리소스 관리
- **백프레셔 처리**: 스트림 레벨에서의 흐름 제어

2. 메모리 최적화

- **Zero-copy**: 가능한 경우 데이터 복사 최소화
- **버퍼 크기 조정**: 네트워크 조건에 맞는 버퍼 크기 설정
- **스트림 제한**: 최대 동시 스트림 수 제한

3. 네트워크 최적화

- **TCP_NODELAY**: 지연 시간 최소화를 위한 Nagle 알고리즘 비활성화
- **포트 재사용**: NAT 환경에서의 연결 성공률 향상
- **인터페이스 감시**: 네트워크 변경 시 자동 주소 업데이트

결론

Rust libp2p의 연결 설정 및 업그레이드 과정은 다음과 같은 특징을 가집니다:

1. **계층적 구조**: Transport → Security → Multiplexing → Application 순서의 명확한 계층
2. **타입 안전성**: 컴파일 타임에 업그레이드 체인 검증
3. **비동기 처리**: 모든 단계에서 논블로킹 비동기 처리
4. **확장성**: 새로운 프로토콜과 업그레이드 쉽게 추가 가능
5. **견고성**: 포괄적인 오류 처리와 복구 메커니즘

이러한 설계는 다양한 네트워크 환경에서 안정적이고 효율적인 P2P 연결을 가능하게 하며, 개발자가 복잡한 네트워킹 세부사항을 신경 쓰지 않고도 고수준의 P2P 애플리케이션을 구축할 수 있게 해줍니다.